

Structured Sparsity in Spiking Neural Networks

G2G Connectivity · Edge Ablation · Dynamic Sparse Training · Alexandros Papafragkakis

	Abstract & Introduction
	PART I — G2G Connectivity in Spiking MLPs
01	Spiking MLP Architecture & Grouping Strategies
02	Experiments on Fashion-MNIST
03	Scaling to CIFAR Datasets — Static Sparsity
04	Dynamic Sparse Training
05	DST Results: CIFAR-10 & CIFAR-100
06	Firing Rate Analysis
07	High Inter-Group Connectivity ($p'=0.90$)
08	ResNet–SNN Extension
09	Discussion — Part I
	PART II — Edge Ablation in Spiking CNNs
10	Spiking CNN Architecture
11	Convolution as a Graph: Edges and Ablation
12	Pruning Strategies Across Training Phases
13	Edge Importance and Low-Importance Criteria
14	Effect of Sparsity on Spiking CNNs — CIFAR-10
15	Layerwise Firing Rate Analysis
16	Discussion — Part II & Unified Conclusions

Abstract

This report presents a unified investigation into structured sparsity for Spiking Neural Networks (SNNs), spanning two complementary approaches that share the same underlying principle: organising connectivity into modular groups reduces redundancy, improves accuracy, and decreases energy consumption relative to dense fully-connected baselines. Part I studies Group-to-Group (G2G) connectivity in spiking MLPs, evaluating three neuron grouping strategies — Index, Mixer, and Random — across Fashion-MNIST, CIFAR-10, and CIFAR-100, with and without Dynamic Sparse Training (DST). Part II extends the framework to spiking convolutional networks, treating inter-channel convolutional kernels as graph edges and evaluating three pruning strategies — pre-training, dynamic, and post-training — across sparsity levels from 30% to 95% on CIFAR-10. Across both parts, structured sparse models consistently outperform parameter-matched dense baselines, dynamic adaptation of connectivity yields additional gains, and firing rates are reduced by a factor of two to three relative to dense architectures. Together, the results establish group-based modularity and dynamic sparse topology adaptation as broadly applicable principles for energy-efficient spiking neural network design.

Introduction

Spiking Neural Networks (SNNs) are increasingly recognised as a promising paradigm for energy-efficient machine learning, drawing inspiration from the sparse, event-driven communication of biological neurons. Unlike conventional ANNs, which propagate continuous-valued activations at every layer, SNNs transmit binary spike signals only when a neuron's membrane potential exceeds a threshold, making computational cost proportional to the number of spike events rather than the number of multiply-accumulate operations.

A central open question is how to structure the connectivity of an SNN to maximise both accuracy and efficiency. This report addresses the question from two complementary angles. Part I examines **Group-to-Group (G2G) connectivity** in fully-connected spiking layers, where neurons are partitioned into groups with full intra-group connectivity and controlled inter-group communication. Part II examines the same principle in convolutional layers, where it manifests as the selective removal of inter-channel kernel connections — modelled as edges in a bipartite graph — according to importance criteria.

Both parts of the report draw on the same theoretical motivation: that biological neural circuits are neither fully connected nor randomly sparse, but organised into functional modules with selective long-range connections. The experiments test whether this organisational principle translates into practical computational benefits when applied to artificial spiking networks trained on image classification tasks.

PART I

G2G Connectivity in Spiking MLPs

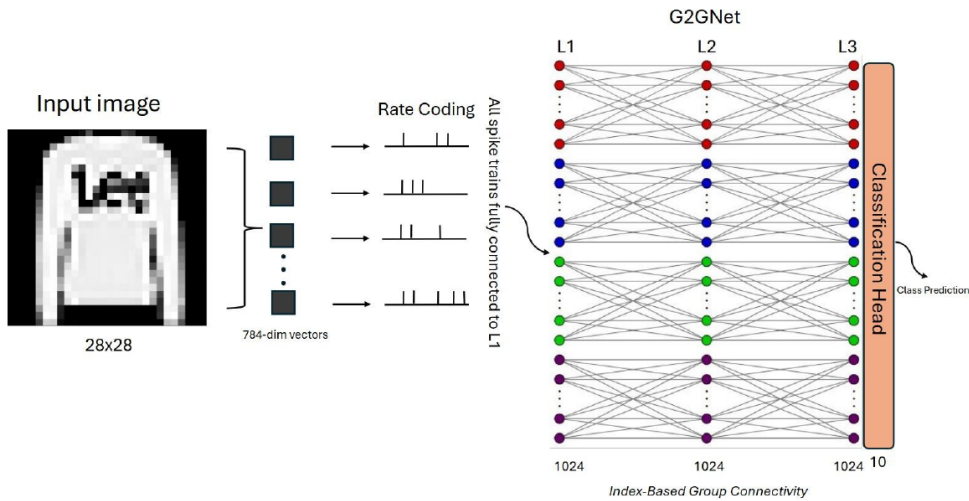
Fashion-MNIST · CIFAR-10 · CIFAR-100 · Dynamic Sparse Training · ResNet Extension

01 Spiking MLP Architecture & Grouping Strategies

The base architecture is a **Spiking MLP** with three hidden LIF (Leaky Integrate-and-Fire) layers (1024→1024→1024 neurons), followed by a 10-neuron output layer. Input images are converted to spike trains using Poisson-like rate encoding over $T=50$ timesteps. Four variants are compared — Dense, Index, Random, Mixer — sharing the same topology but applying different sparse connectivity masks to the hidden layers. Each layer is partitioned into $G=8$ groups. Intra-group connections are always fully active ($p=1.0$), while inter-group connections are controlled by $p' \in [0, 1]$, giving: Active = $N_{in} \times N_{out} \times (0.125 + 0.875 \times p')$.

The **Index** strategy assigns neurons to contiguous blocks, preserving spatial locality. **Mixer** alternates index-based grouping (even layers) with interleaved grouping (odd layers: group k contains neurons l where $k = l \bmod G$), enabling structured cross-group feature integration. **Random** assigns neurons without regard to position — a sparsity-matched unstructured baseline.

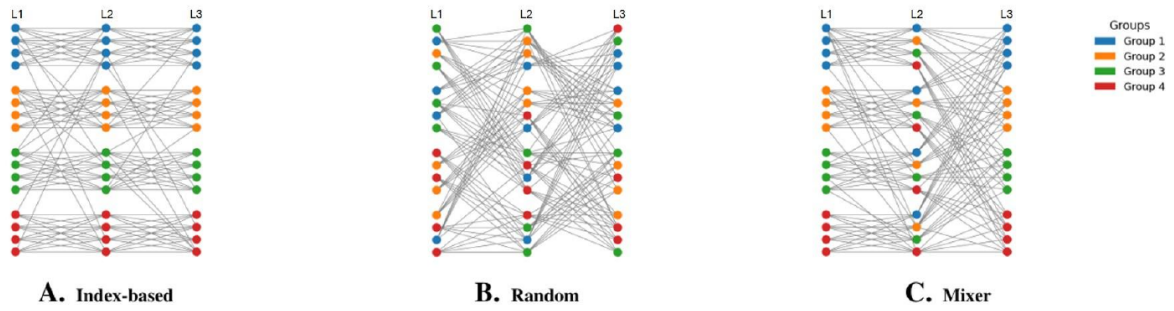
Spiking MLP Architecture:



Spiking MLP architecture with three hidden LIF layers. The input image is converted into spike trains using a *Poisson-like rate encoding* scheme, where each pixel intensity defines the **firing probability** and is **sampled independently across T timesteps**. The diagram illustrates the **index** version of the network; **dense**, **Random**, and **Mixer** variants share the same architecture but differ in how **sparse connectivity masks** are applied to the hidden layers.

Figure 1. G2GNet with index-based group connectivity across three LIF hidden layers.

Neuron Grouping Strategies



- **A. Index-based:** neurons are divided into contiguous blocks by index, preserving spatial locality from the input image across layers.
- **B. Random:** neurons are assigned to groups without regard to position, disrupting spatial structure and coherence.
- **C. Mixer:** groups alternate between index-based (even-numbered layers) and interleaved assignments (odd-numbered layers, where group k contains all neurons i such that $k = i \bmod G_i$). This approach maintains spatial locality in some layers while promoting integration across spatial regions in others, balancing modularity and feature mixing

Figure 2. Grouping strategies: A. Index-based · B. Random · C. Mixer.

02 Experiments on Fashion-MNIST

Models are trained for 20 epochs with Adam ($lr=1e-3$, $wd=1e-4$). The Dense baseline uses 447 hidden units matching the $\sim 754k$ parameter budget. All sparse architectures outperform the Dense baseline (87.04%) at every p' . Index peaks at $p'=0.05$ (89.52%); Mixer at $p'=0.10$ (89.42%). Accuracy is stable across the full p' range, indicating that the inductive bias of structured grouping — rather than the precise degree of inter-group communication — drives the accuracy advantage.

Firing rate analysis reveals a complementary efficiency benefit. Spiking activity decreases monotonically $LIF1 \rightarrow LIF2 \rightarrow LIF3$ in all configurations. Increasing p' reduces overall firing rates by $\sim 45\text{--}50\%$ as p' rises from 0.00 to 0.50, while test accuracy remains stable at $\sim 87\text{--}88\%$ — a strong energy–accuracy decoupling. 5-fold cross-validation shows Index is most stable at $p'=0.25$, Mixer at $p'=0.00$, and Random displays the highest variability across splits.

Best Test Accuracy per Connectivity Strategy

p'	Index (%)	Random (%)	Mixer (%)	Dense (%)
0.00	88.32	88.11	89.18	—
0.05	89.52	88.31	88.99	—
0.10	89.20	88.50	89.42	—
0.15	89.40	87.94	88.82	—
0.20	89.01	88.42	89.06	—
0.25	88.83	88.73	89.13	—
1.00	—	—	—	87.04

Best test accuracy for each connectivity strategy as a function of the inter-group probability p' . For each configuration, the reported value corresponds to the highest test accuracy achieved during training. The dense baseline ($p' = 1.00$) uses 447 hidden units to match the parameter budget of the sparse architectures ($\sim 754k$ parameters).

Figure 3. Best test accuracy per strategy vs p' .

Effect of Inter-Group Connectivity on Firing Rates:

p'	Cross-Group Ratio	Input Entropy	Within-Group Corr	Coherence	Firing Rate	Test Acc
0.00	–	0.000	0.0104	–	0.0222	87.78%
0.10	0.411	2.133	0.0073	0.032	0.0161	88.55%
0.25	0.636	2.733	0.0052	0.001	0.0109	88.47%
0.50	0.778	2.948	0.0042	0.000	0.0085	87.68%

Effect of inter-group connectivity (p') on cross-group ratio, temporal coherence, and firing rate. Higher values of p' are associated with increased cross-group inputs and reduced temporal synchrony and spiking activity.

Figure 4. Cross-group ratio, entropy, coherence, firing rate and accuracy vs p' .

03 Scaling to CIFAR Datasets — Static Sparsity

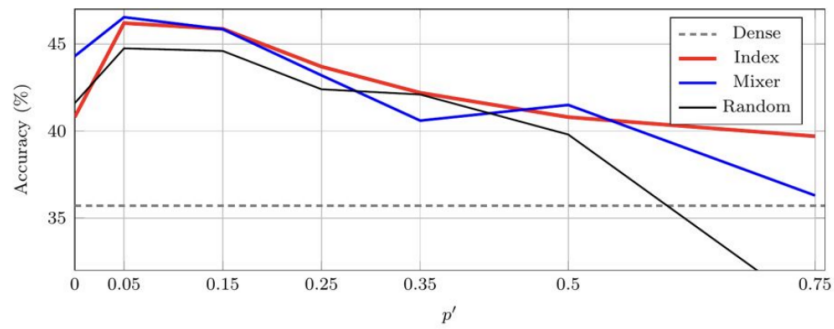
On CIFAR-10 and CIFAR-100, sparse models beat the Dense baseline (35.7% / 11.1%) at every sparsity level. At $p'=0.00$, Mixer achieves 44.3% on CIFAR-10 versus 35.7% for Dense — a gain of over eight percentage points with fewer connections. The optimal range $p' \in [0.05, 0.15]$ offers the best accuracy–efficiency trade-off. High sparsity ($p'=0.75$) causes degradation, especially on CIFAR-100, where under-connected representations lose fine-grained discriminative capacity.

Performance of Static G2GNet Across Sparsity Levels:

Model	p'	CIFAR-10		CIFAR-100	
		Acc. (%)	FR	Acc. (%)	FR
Dense (baseline)	—	35.7	0.0114	11.1	0.0218
Index	0.00	40.8	0.0159	16.89	0.013589
Mixer	0.00	44.3	0.0236	20.40	0.024984
Random	0.00	41.6	0.0146	18.97	0.014688
Index	0.05	46.2	0.0123	19.17	0.014485
Mixer	0.05	46.54	0.0146	21.31	0.015635
Random	0.05	44.75	0.0096	18.44	0.013198
Index	0.15	45.87	0.0085	18.51	0.009517
Mixer	0.15	45.85	0.0080	19.33	0.009972
Random	0.15	44.60	0.0071	18.30	0.008266
Index	0.25	43.7	0.0078	17.28	0.008815
Mixer	0.25	43.2	0.0075	16.64	0.005449
Random	0.25	42.4	0.0064	16.95	0.007127
Index	0.35	42.2	0.0055	16.87	0.006702
Mixer	0.35	40.6	0.0055	16.29	0.006886
Random	0.35	42.1	0.0063	16.65	0.008080
Index	0.50	40.8	0.0079	13.58	0.005994
Mixer	0.50	41.5	0.0069	14.52	0.006820
Random	0.50	39.8	0.0048	15.53	0.008331
Index	0.75	39.7	0.0068	10.50	0.007840
Mixer	0.75	36.3	0.0049	13.93	0.009655
Random	0.75	29.6	0.0040	12.86	0.009121

Figure 5. Accuracy and firing rate per model and p' on CIFAR-10 and CIFAR-100.

CIFAR-10



CIFAR-100

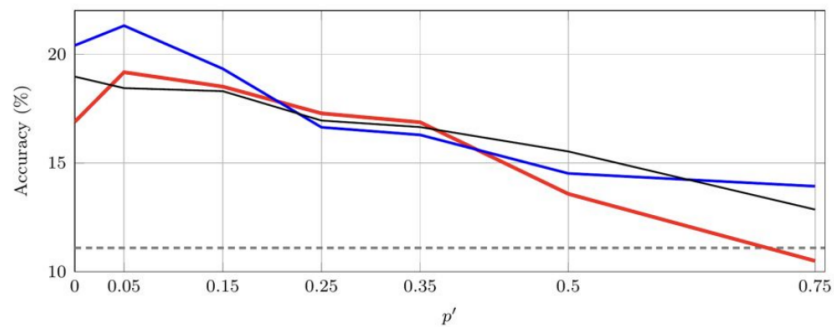


Figure 6. Accuracy vs p' curves — CIFAR-10 (top) and CIFAR-100 (bottom).

04 Dynamic Sparse Training

DST maintains a fixed sparsity budget by periodically **pruning** low-importance connections and **growing** the same number of new ones during training. Three pruning criteria are evaluated: magnitude (smallest weights), random removal, and Hebbian (low co-activation). Two growth criteria: random inactive edges, or high Hebbian co-activation similarity. This mirrors biological **structural plasticity** — the brain adapts its wiring under energy constraints, reinforcing useful synapses and eliminating weak ones during learning.

Dynamic Sparse Training – Pruning & Growth Methods

DST maintains a sparse architecture by periodically **pruning** active connections according to specific criteria—removing either the smallest-magnitude weights, randomly selected edges, or connections with low pre–post Hebbian co-activation.

An **equal** number of new connections are then introduced through **growth**, chosen either at random or based on high Hebbian similarity, allowing the network to preserve fixed sparsity while continuously adapting its connectivity.

Pruning (C_P)	
Method	Criterion
Magnitude	Smallest weights
Random	Random removal
Hebbian	Low co-activation

Growth (C_G)	
Method	Criterion
Random	Random inactive edges
Hebbian	High co-activation

Overview of DST Pruning and Growth Strategies

Figure 7. Overview of DST pruning (left) and growth (right) strategies.

05 DST Results: CIFAR-10 & CIFAR-100

DST gains +0.5–2.7% over static on CIFAR-10 and up to +1.5% on CIFAR-100. All architectures peak at $p'=0.05$ (Index: 47.10%, Mixer: 47.07%, Random: 45.33%). DST most dramatically rescues the Random architecture at high sparsity: $p'=0.75$ improves from 29.6% to 39.7% (+10.1%), demonstrating that adaptive rewiring compensates for the absence of structural inductive bias. On CIFAR-100 Mixer consistently leads, while on CIFAR-10 Index and Mixer trade places.

Accuracy vs Inter-group Connectivity (p'): Dense, Static, and DST Sparsity:

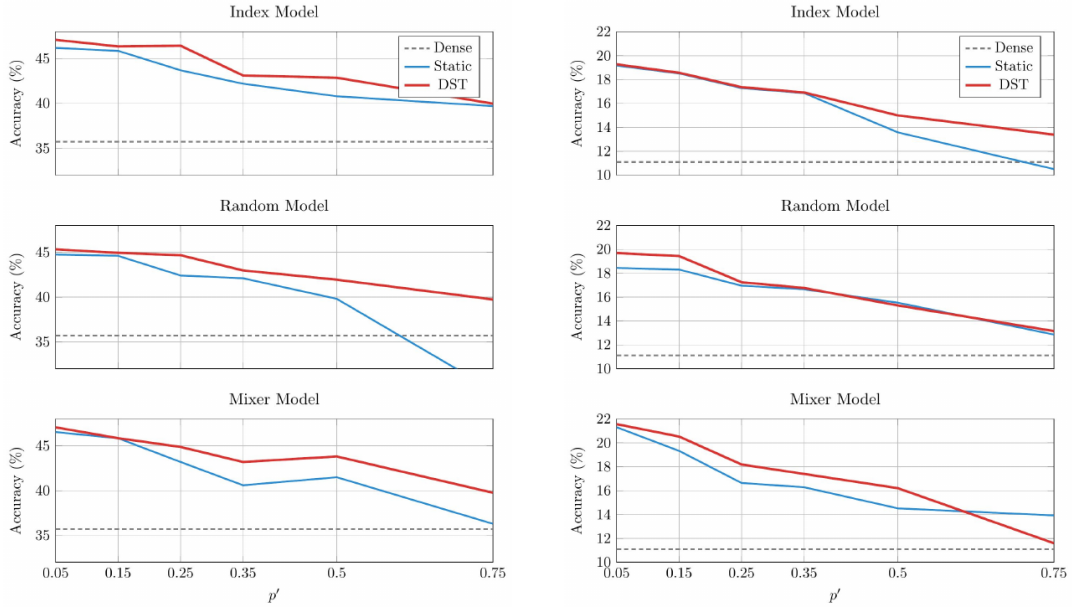


Figure 8. Dense / Static / DST accuracy vs p' — all three models.

DST Performance: Index vs Random vs Mixer

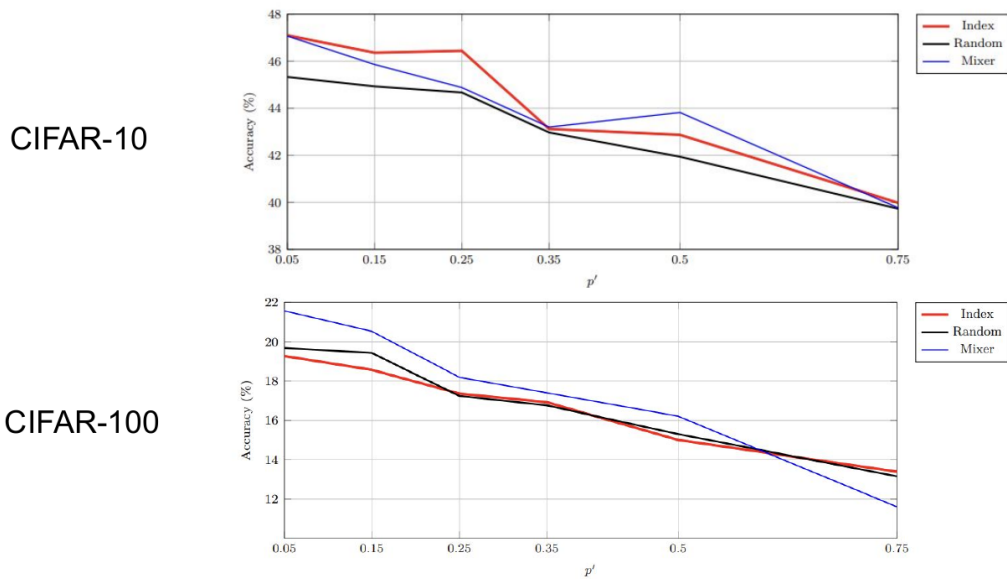


Figure 9. DST head-to-head: Index vs Random vs Mixer.

06 Firing Rate Analysis

DST achieves lower firing rates than static sparsity while maintaining equal or higher accuracy. Both sparse regimes fire far less than Dense across all p' values. At $p'=0.90$, Index and Mixer converge toward Dense-like behaviour — the grouping inductive bias progressively disappears as $p' \rightarrow 1$. DST still adds value even here: Random DST reaches 40.03% vs Dense 35.70% on CIFAR-10.

Firing Rate Comparison Across Connectivity Regimes:

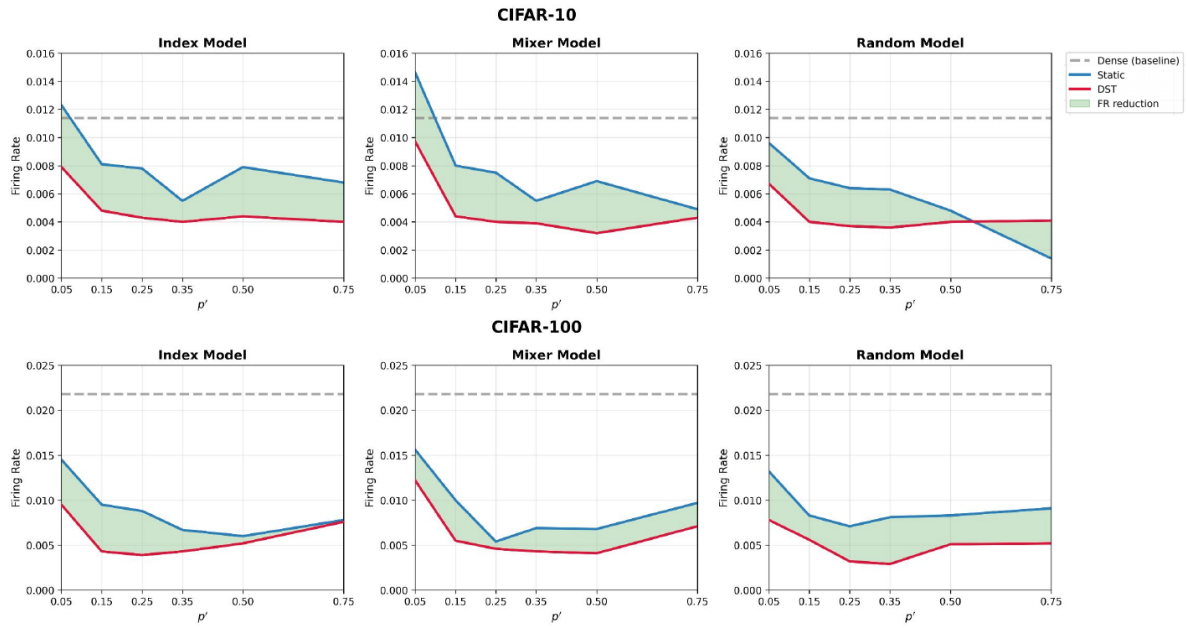


Figure 10. Static vs DST firing rates — Index, Mixer, Random on CIFAR-10 and CIFAR-100.

07 High Inter-Group Connectivity ($p'=0.90$)

Effect of High Inter-Group Connectivity ($p'=0.90$)

Model	Method	p'	Acc. (%)	FR
Fully Connected (dense)	–	–	35.70	0.0114
Index	Static	0.90	36.24	0.0059
Random	Static	0.90	37.31	0.0078
Mixer	Static	0.90	36.67	0.0049
Index	DST (R/R)	0.90	37.77	0.0075
Random	DST (R/R)	0.90	40.03	0.0084
Mixer	DST (H/R)	0.90	39.14	0.0045

Dataset: CIFAR-10

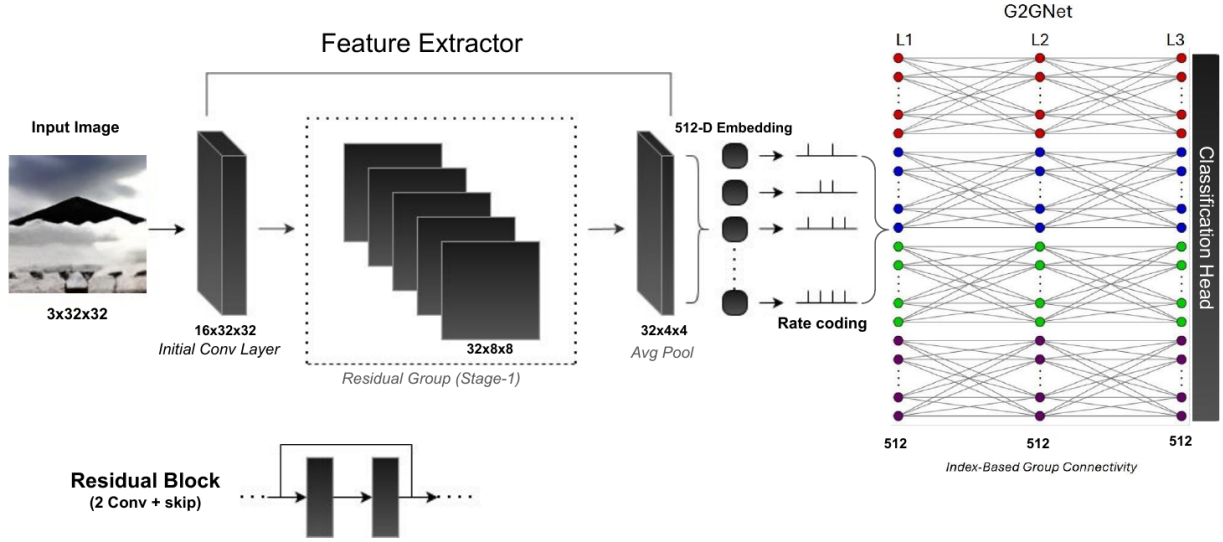
Model	Method	p'	Acc. (%)	FR
Fully Connected (dense)	–	–	11.10	0.0218
Index	Static	0.90	11.35	0.0075
Random	Static	0.90	11.30	0.0095
Mixer	Static	0.90	11.32	0.0091
Index	DST (H/R)	0.90	11.40	0.0087
Random	DST (R/H)	0.90	11.63	0.0110
Mixer	DST (H/R)	0.90	12.68	0.0087

Dataset: CIFAR-100

Figure 11. Static and DST results at $p'=0.90$ — CIFAR-10 (top) and CIFAR-100 (bottom).

08 ResNet-SNN Extension

A **truncated ResNet-32 (Block-1)** extracts 512-D embeddings from CIFAR images. Embeddings are min-max normalised, rate-encoded, and classified by a 3-layer G2G LIF network (512→512→512). Trained 50 epochs, $T=50$. Sparse models match Dense accuracy (~93.4–93.5% on CIFAR-10) while cutting firing rates 2–3×. Accuracy saturates early — firing rate becomes the primary differentiator. Index achieves the lowest firing rates; Mixer peaks at intermediate p' .



Overview of the proposed pipeline. A truncated ResNet-32 (Block-1) is used as a feature extractor, followed by adaptive average pooling to produce a 256-D feature embedding. The embedding is min-max normalized and converted into spike trains using rate coding, then fed into a three-layer G2G spiking neural network with structured group-based connectivity for classification.

Figure 12. Truncated ResNet-32 extractor + G2G SNN classifier pipeline.

CIFAR-10:

Performance–efficiency trade-off on CIFAR-10. Across different values of p' , dense and structured sparse SNNs reach similar test accuracy, whereas index-, mixer-, and random-based connectivity substantially reduce firing rates relative to the dense model.

Training Details

- **Epochs:** 50
- **Time steps (T):** 50
- **Batch size:** 256
- **Optimizer:** Adam (learning rate = $1e-3$, weight decay = $1e-4$)
- **Loss function:** Cross-Entropy

p_{inter}	Model	Best Test Acc.	Firing Rate	Num Params
0.00	Dense*	0.9344	0.2159	1,338,783
	Index	0.9347	0.0590	1,337,362
	Mixer	0.9343	0.0924	1,337,362
	Random	0.9339	0.0892	1,337,362
0.05	Dense*	0.9345	0.2204	1,795,227
	Index	0.9346	0.0720	1,796,112
	Mixer	0.9348	0.0974	1,796,112
	Random	0.9347	0.0857	1,796,112
0.15	Dense*	0.9348	0.1881	2,713,288
	Index	0.9349	0.0766	2,713,617
	Mixer	0.9350	0.0852	2,713,617
	Random	0.9345	0.0632	2,713,617
0.25	Dense*	0.9349	0.1611	3,631,737
	Index	0.9345	0.0736	3,631,122
	Mixer	0.9351	0.0757	3,631,122
	Random	0.9336	0.0757	3,631,122
0.35	Dense*	0.9345	0.1634	4,548,927
	Index	0.9349	0.0655	4,548,624
	Mixer	0.9347	0.0762	4,548,624
	Random	0.9345	0.0799	4,548,624
0.50	Dense*	0.9351	0.1600	5,922,537
	Index	0.9337	0.0823	5,924,882
	Mixer	0.9343	0.0952	5,924,882
	Random	0.9351	0.0816	5,924,882
0.75	Dense*	0.9342	0.1283	8,215,338
	Index	0.9346	0.1048	8,218,642
	Mixer	0.9349	0.1019	8,218,642
	Random	0.9336	0.1093	8,218,642
1.00	Dense*	0.9340	0.1099	10,512,394

*Dense models are fully connected (no sparsity); they are capacity-matched to sparse models at each p_{inter} value but do not depend on p_{inter} themselves.

Figure 13. Performance–efficiency trade-off on CIFAR-10.

09 Discussion — Part I

Why Structured Sparsity Outperforms Dense Connectivity

Dense connectivity causes all neurons to receive highly correlated inputs, reducing representation diversity. Structured sparsity breaks this homogeneity by enforcing distinct input sets per group, allowing specialisation. The result is a more diverse, information-rich set of representations with fewer parameters.

Why Index Wins on CIFAR-10, Mixer on CIFAR-100

CIFAR-10's 10 broadly separated classes are classifiable from local feature patterns — Index's strong spatial locality keeps representations clean and easily separable. CIFAR-100's 100 fine-grained classes require combining subtle features from different spatial locations — precisely what Mixer facilitates through its alternating grouping pattern.

The Optimal Connectivity Regime and Why DST Works

Performance peaks at $p' \in [0.05, 0.15]$ — groups share information through a small number of inter-group bridges without losing modularity. DST improves on static sparsity by committing connectivity decisions after training has revealed which connections matter, producing topologies that are shaped by the data rather than random initialisation.

Edge Ablation in Spiking CNNs

Convolutional layers as graphs · Pre / Dynamic / Post-training pruning · CIFAR-10

10 Spiking CNN Architecture

The model is **KS32_FullySpiking_Small**: seven Convolutional Spiking Layers (CSL1–CSL7), each with a convolutional layer, batch normalisation, and a spiking neuron activation. Residual connections link selected non-adjacent blocks. A 4×4 adaptive average pooling layer aggregates spatial feature maps, followed by two Dense Spiking Layers (DSL1–DSL2) for classification. All activations are binary spikes, making the architecture fully compatible with event-driven neuromorphic execution.

Spiking Convolutional Neural Network Architecture

The model contains seven Convolutional Spiking Layers (CSL1–CSL7), each consisting of a convolutional layer, batch normalization, and a spiking neuron layer, with residual connections between selected blocks. Feature maps are aggregated using a 4×4 adaptive average pooling layer and processed by two Dense Spiking Layers (DSL1–DSL2), forming the spiking neural network classifier.

CSL1–CSL7 = Convolutional Spiking Layers 1–7
 DSL1–DSL2 = Dense Spiking Layers 1–2
 Conv Layer = Convolutional Layer
 BatchNorm = Batch Normalization
 Spiking Layer = Spiking Neuron Layer
 Adaptive AvgPool = Adaptive Average Pooling Layer

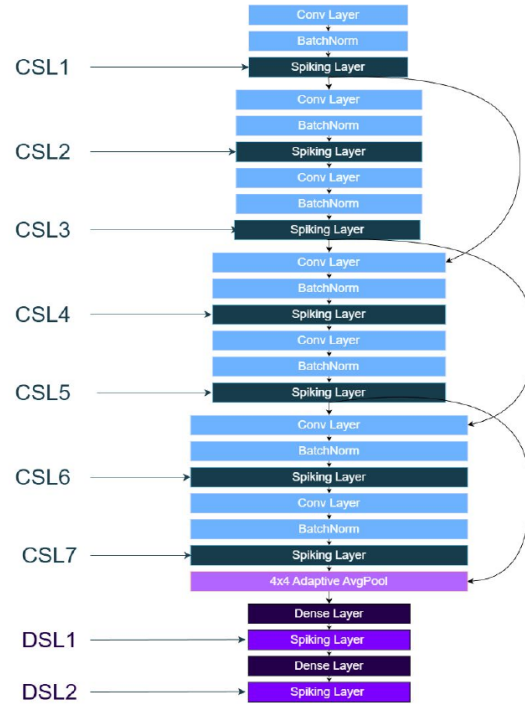


Figure 14. KS32_FullySpiking_Small: seven CSLs with residual connections and two dense spiking classifier layers.

11 Convolution as a Graph: Edges and Ablation

A convolutional layer is a directed bipartite graph: input channels c are source nodes, output channels c' are target nodes, and edge $c \rightarrow c'$ is parameterised by kernel $K(c', c)$. The edge contribution is $E(c \rightarrow c')(x, y) = \sum_i \sum_j I_c(x+i, y+j) \cdot K(c', c)(i, j)$, and the full output is $O(c') = \sum_c (I_c * K(c', c))$. **Ablating** an edge means setting $K(c', c) = 0$, zeroing channel c 's contribution to c' . If other edges carry redundant information the ablation has little effect; if the edge was unique, performance degrades. This graph perspective unifies structured grouped convolution, magnitude-based pruning, and dynamic sparse training into a single conceptual framework.

12 Pruning Strategies Across Training Phases

Pre-training pruning randomly deactivates a fixed percentage of edges before training. The network trains within this fixed sparse topology — efficient but uninformed. **Dynamic pruning** starts the same way but periodically re-evaluates edge importance, removing the least useful connections and reactivating an equal number of inactive ones, maintaining constant sparsity while adapting topology throughout training. **Post-training pruning** trains a dense network to convergence, then removes low-importance edges and fine-tunes to recover performance. This makes importance decisions on fully-trained weights but suffers a structural mismatch at high sparsity that fine-tuning cannot bridge.

Pruning Strategies Across Training Phases

- **Pre-training pruning:**
We start from a partially connected convolutional layer by randomly deactivating a fixed percentage of edges $c \rightarrow c'$, establishing an initial sparse connectivity pattern.
- **Dynamic pruning during training:**
Training starts with a sparse convolutional layer, created by randomly deactivating a fixed percentage of connections. During training, the importance of active connections is checked at regular intervals, and some of the least important ones are removed. To keep the same level of sparsity, the same number of inactive connections is randomly added back, allowing the network to change its connectivity over time.
- **Post-training pruning:**
The network is first trained with a fixed connectivity pattern. After training, a fixed percentage of low-importance edges is removed, followed by a fine-tuning phase to recover performance

Figure 15. The three pruning strategies: pre-training, dynamic, and post-training.

13 Edge Importance and Low-Importance Criteria

Edge importance is measured by the **L1 norm of the associated kernel**: $\text{importance}(c \rightarrow c') = \|K(c', c)\|_1 = \sum_{i,j} |K(c', c)(i, j)|$. A near-zero L1 norm means all kernel weights are negligible and the edge contributes minimally to the output — a safe candidate for removal. A large L1 norm signals strong influence on the output and the edge should be preserved.

Low-importance edges in convolutional layers

$$\text{importance}(c \rightarrow c') = \|K_{c',c}\|_1$$

- $c \rightarrow c'$: the edge from input channel c to output channel c' .
- $K_{c',c}$: the convolutional kernel associated with this connection.
- $\|\cdot\|_1$: the L1 norm, defined as the sum of the absolute values of all elements.

If $\|K_{c',c}\|_1 \approx 0$:

- all kernel weights are close to zero,
- the kernel has a negligible effect on the output feature maps,
- the corresponding edge is considered low-importance.

If $\|K_{c',c}\|_1$ is large:

- the kernel has a strong influence on the output feature maps,
- the corresponding edge is considered important.

Figure 16. Edge importance = $\|K(c', c)\|_1$. Low norm $\rightarrow$ negligible contribution $\rightarrow$ candidate for pruning.

14 Effect of Sparsity on Spiking CNNs — CIFAR-10

Experiments use KS32_FullySpiking_Small with $T=4$, 50 epochs, SGD (momentum 0.9, $wd=5 \times 10^{-4}$), cosine annealing. Dense baseline: 83.37% / FR=0.1197. Pre-training pruning degrades gracefully: 82.19% at 30%, 71.44% at 90%. Dynamic pruning consistently outperforms it: 82.46%, 72.50%. Post-training pruning collapses catastrophically: 16.80% at 70%, near-chance at 90–95%. This collapse reflects the fundamental mismatch between weights trained for dense connectivity and the topology imposed at high sparsity — fine-tuning cannot recover.

Effect of Sparsity Strategies on Spiking Convolutional Networks

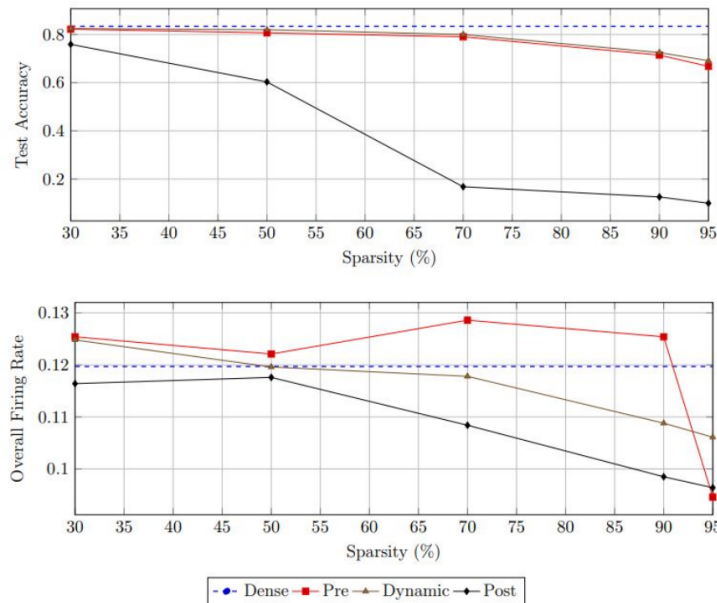
CIFAR 10:

Training Configuration

- **Epochs:** 50
- **Simulation Time Steps:** $T=4$
- **Batch Size:** 256
- **Optimizer:** SGD
Momentum: 0.9
Weight Decay: 5×10^{-4}
- **Learning Rate Scheduler:** Cosine Annealing
- **Loss Function:** Cross-Entropy Loss

Strategy	Sparsity	Test Accuracy	Overall FR
Dense	0.00	0.8337	0.1197
Pre	30.00	0.8219	0.1254
Dynamic	30.00	0.8246	0.1248
Post	30.00	0.7590	0.1164
Pre	50.00	0.8068	0.1221
Dynamic	50.00	0.8198	0.1196
Post	50.00	0.6030	0.1176
Pre	70.00	0.7904	0.1286
Dynamic	70.00	0.7999	0.1178
Post	70.00	0.1680	0.1084
Pre	90.00	0.7144	0.1254
Dynamic	90.00	0.7250	0.1088
Post	90.00	0.1257	0.0985
Pre	95.00	0.6676	0.0946
Dynamic	95.00	0.6911	0.1061
Post	95.00	0.0996	0.0964

Figure 17. Test accuracy and firing rate per pruning strategy and sparsity level.



Test accuracy (top) and overall firing rate (bottom) as a function of sparsity level for different training strategies. The dense network is shown as a dashed baseline, while Pre, Dynamic, and Post correspond to static sparsity, dynamic sparsity with rewiring, and post-training pruning, respectively.

Figure 18. Accuracy (top) and firing rate (bottom) vs sparsity.

15 Layerwise Firing Rate Analysis

CSL7 consistently exhibits the highest firing rates — often above the dense baseline — because it must integrate information regardless of sparsity, and fewer incoming connections concentrate activation on the remaining channels. Dynamic pruning produces smooth, gradually declining layerwise profiles. Pre-training pruning shows irregular patterns with compensatory over-activation. Post-training pruning shows collapsing rates at high sparsity — not efficiency, but failure. The smoothness of the layerwise firing profile is a more reliable indicator of computational health than the aggregate firing rate alone.

Layerwise Firing Rates Across Sparsity Strategies

Strategy	Sparsity	CSL1	CSL2	CSL3	CSL4	CSL5	CSL6	CSL7
Dense	0.00	0.1223	0.0682	0.1740	0.0862	0.1096	0.0454	0.2606
Pre	30.00	0.1255	0.0693	0.1811	0.0982	0.1237	0.0499	0.2600
Pre	50.00	0.1096	0.0699	0.1700	0.1119	0.1357	0.0523	0.2515
Pre	70.00	0.1120	0.0675	0.1854	0.1294	0.1471	0.0605	0.2420
Pre	90.00	0.0923	0.0564	0.1938	0.1020	0.1899	0.0744	0.2291
Pre	95.00	0.0924	0.0217	0.1107	0.0805	0.1727	0.0756	0.2218
Dynamic	30.00	0.1244	0.0717	0.1930	0.0889	0.1105	0.0437	0.2482
Dynamic	50.00	0.1151	0.0630	0.1735	0.0937	0.1318	0.0490	0.2459
Dynamic	70.00	0.1070	0.0603	0.1654	0.1046	0.1483	0.0531	0.2310
Dynamic	90.00	0.0904	0.0376	0.1624	0.0964	0.1600	0.0618	0.2231
Dynamic	95.00	0.1029	0.0426	0.1493	0.0715	0.1541	0.0670	0.2115
Post	30.00	0.1079	0.0636	0.1704	0.0998	0.1159	0.0422	0.2545
Post	50.00	0.1159	0.0664	0.1757	0.0915	0.1004	0.0416	0.2593
Post	70.00	0.1145	0.0625	0.1405	0.1044	0.1121	0.0450	0.2038
Post	90.00	0.1237	0.0439	0.1265	0.0934	0.1057	0.0438	0.1538
Post	95.00	0.1162	0.0329	0.1537	0.1047	0.0860	0.0306	0.1120

Figure 19. Layerwise firing rates (CSL1–CSL7) across sparsity levels for Pre, Dynamic, and Post.

16 Discussion — Part II & Unified Conclusions

Why Dynamic Pruning Outperforms Pre-Training and Post-Training

Pre-training pruning commits to topology before any task signal is seen. Dynamic pruning defers the decision, allowing gradient information to reveal which connections matter before any are permanently removed. Post-training pruning makes well-informed decisions on fully-trained weights, but removes too many connections simultaneously at high sparsity — a structural shock that fine-tuning cannot undo. Dynamic pruning's incremental changes give weights time to adapt at each step, avoiding both problems.

The Unified Principle Across Both Parts

Both parts of this report arrive at the same core conclusion from different directions. Part I shows that organising fully-connected layers into groups with controlled inter-group communication improves accuracy and reduces firing rates. Part II shows that treating convolutional inter-channel connections as graph edges and selectively removing the least important ones achieves the same twin benefit. In both cases the key ingredient is **structured inductive bias**: by organising connectivity around coherent feature subsets rather than allowing arbitrary connections, the network is guided toward representations that are more informative and more efficient to compute.

Dynamic Adaptation as a Shared Mechanism

DST in Part I and dynamic pruning in Part II are the same idea applied to different layer types. Both maintain a fixed connectivity budget while continuously adapting topology during training, and both consistently outperform their static counterparts. This convergence across architectures and layer types suggests that activity-dependent structural reorganisation is a broadly applicable principle — not an architecture-specific trick — and that it mirrors the biological mechanism of structural plasticity at a meaningful computational level.

Limitations and Future Directions

Several important limitations should be noted. All experiments use rate coding, the simplest encoding scheme; temporal or phase-based coding could reduce firing rates further and may interact differently with group structure. The LIF neuron model is a significant simplification of biological neurons. Evaluation on more complex datasets (ImageNet-scale) remains unexplored. Future work should investigate temporal coding schemes, more expressive neuron models, and the combination of G2G structured connectivity in both fully-connected and convolutional layers within a single end-to-end spiking architecture.